DG INTERNS HUB — TECHNICAL STUDY WRITEUP

WEEK 4 – SIEM & SECURITY MONITORING: SPLUNK EXPLORING SPL

Prepared By:	**Gunjan**	**Department:**	
Internship:	**DG Interns Hub Cybersecurity**	**TryHackMe Room:**	
Submission Date:	**31 October 2026**	**Output File:**	

TABLE OF CONTENTS

1. Overview & Study Objectives
2. SPL Architecture & Pipeline Concept
3. Fundamental Filtering Commands
4. Data Transformation Commands (`stats`)
5. Field Extraction & Evaluation (`eval`, `rex`)
6. Time-Series Analysis (`timechart`)
7. SOC Threat Hunting Use Cases
8. Key Learnings & Conclusion

1. OVERVIEW & STUDY OBJECTIVES

Splunk Search Processing Language (SPL) is a powerful query language used to search, filter, aggregate, modify, and visualize security data ingested into Splunk. SOC analysts utilize SPL daily to conduct threat hunting, investigate security alerts, perform root-cause analysis, and construct operational dashboards.

This writeup details the concepts, query syntax, command pipelines, and forensic workflows taught in TryHackMe's *Splunk: Exploring SPL* room.

2. SPL ARCHITECTURE & PIPELINE CONCEPT

SPL operates on a sequential pipeline model using the pipe character (`|`). A query starts with an initial search term to retrieve raw data blocks from disk indexers. Each subsequent pipe takes the output of the previous command, transforms or filters the dataset in memory, and passes it to the next stage.

```
# Basic Pipeline Structure Example:
index=main sourcetype=access_combined status=200
| stats count by clientip
| where count > 50
| sort - count
```

3. FUNDAMENTAL FILTERING COMMANDS

- `search`: Explicit command (or implied at query start) to retrieve events matching specific strings or field-value pairs (e.g., `EventCode=4625`).
- `fields`: Controls which fields are loaded into memory. Specifying `| fields + _time, user, src_ip` speeds up query execution by excluding unused fields early in the pipeline.
- dedup: Removes duplicate events based on designated field combinations (e.g., `| dedup src_ip` keeps only the most recent event per IP).

4. DATA TRANSFORMATION COMMANDS

Transforming commands process individual events into statistical summary tables:

- `stats`: Calculates statistical metrics across grouped fields. Common functions include `count` (total events), `dc()` (distinct count of unique values), `sum()` (total numeric sum), and `avg()` (mean value).

```
# Example: Count distinct users targeted per source IP
index=wineventlog EventCode=4625
| stats count, dc(TargetUserName) as unique_targets by Source_Network_Address
| sort - count
```

- `top` / `rare`: Finds the most or least frequent values of a field, automatically providing count and percentage columns (e.g., `| top limit=5 TargetUserName`).

5. FIELD EVALUATION & REGULAR EXPRESSIONS

When default field extractions are insufficient, SPL provides dynamic field creation and Regex extraction tools:

- `eval`: Computes new fields based on mathematical operations, string concatenations, or conditional statements.

```
# Example: Convert raw bytes to Megabytes and apply threshold flag
index=network_logs action=allowed
| eval MB = round(bytes/1024/1024, 2)
| eval risk_level = if(MB > 500, "HIGH", "NORMAL")
| table src_ip, dest_ip, MB, risk_level
```

- `rex`: Extracts custom fields from unstructured raw log text using Regular Expressions at search time.

```
# Example: Extract custom API key from HTTP URL path
index=web_logs
| rex field=uri_path "api_key=(?<extracted_api_key>[A-Za-z0-9]+)"
| stats count by extracted_api_key
```

6. TIME-SERIES ANALYSIS

Security analysis relies heavily on evaluating events over time to spot traffic spikes or periodic beaconing:

- `timechart`: Renders statistical aggregations into time-binned tables optimized for charts.

```
# Example: Hourly trend of successful vs failed logins
index=wineventlog (EventCode=4624 OR EventCode=4625)
| timechart span=1h count by EventCode
```

- `bin`: Groups timestamps into discrete time windows (e.g., `| bin _time span=15m`) prior to running standard `stats` operations.

7. SOC THREAT HUNTING USE CASES

SPL enables proactive threat hunting across common enterprise attack scenarios:

- **Detecting Password Spray Attacks:** Aggregating failed logins by source IP and filtering for high distinct targeted usernames: `index=wineventlog EventCode=4625 | stats dc(TargetUserName) as targets by Source_Network_Address | where targets > 15`.
- **Detecting Malicious PowerShell Execution:** Querying Sysmon Event Code 1 (Process Creation) for obfuscated flags: `index=sysmon EventCode=1 Image="*powershell.exe*" (CommandLine="*-enc*" OR CommandLine="*-nop*") | table _time, Computer, User, CommandLine`.

8. KEY LEARNINGS & CONCLUSION

Mastering SPL syntax enables analysts to rapidly transform millions of unstructured log entries into actionable security intelligence. Filtering data early, using stats efficiently, and leveraging regex extractions form the foundation of efficient SOC investigations.